

基于 E203 RISC-V 软硬件协同的 INT8 GEMV 加速器设计

陈炯涛董山

2026 年 5 月

目录

0	快速预览简介	3
1	项目背景与总体目标	4
1.1	背景	4
1.2	目标	4
2	开发环境与工程结构	4
2.1	硬件与软件环境	4
2.2	工程结构	5
3	软件基线设计与仿真结果	5
3.1	软件 GEMV kernel	5
3.2	软件基线在整体框架中的作用	6
4	硬件加速器设计	7
4.1	总体方案	7
4.2	寄存器映射	7
4.3	计算数据通路	8
5	SoC 集成与固件设计	8
5.1	E203 外设总线集成	8
5.2	固件 MMIO 访问	9
6	程序镜像转换与关键问题修复	10
6.1	64-bit 程序镜像转换	10
6.2	BRAM 推断失败问题	10

7	上板调试与当前验证结果	11
7.1	ILA 配置	11
7.2	PL UART 接出与串口配置	11
7.3	板端串口结果	12
7.4	Reset 与时钟设计	13
8	性能分析	14
8.1	核心计算与端到端性能	14
8.2	端到端开销	14
9	当前限制与后续工作	15
10	结论	15

0 快速预览简介

项目目标 本项目面向大模型推理中的线性层计算，基于 HummingBird E203 RISC-V 处理器和 ALINX AXU3EG FPGA 开发板，实现一个 8×8 INT8 GEMV 矩阵向量乘加硬件加速器。项目采用先软件基线、再硬件外设、最后上板 ILA 与 PL UART 验证的流程，形成从 E203 仿真到 FPGA 板端运行的完整软硬件协同验证链路。

核心技术

- HummingBird E203 RISC-V SoC, 使用 ICB peripheral bus 接入自定义 memory-mapped 外设
- 8×8 INT8 GEMV 软件基线, 基于 E203 `mcycle` CSR 统计 kernel cycle
- `gemv_accel` 硬件加速器, 采用 8 路并行 signed MAC, 每周期处理一列
- Vivado 2022.2 FPGA 工程迁移, 目标器件为 `xczu3eg-sfvc784-1-i`
- ILA 在线调试, 重点观察 GEMV 外设内部状态、计算列计数和完成信号
- PL UART 串口输出, 获取完整 `y[0..7]` 正确性结果和端到端 cycle 数据

当前重要指标

- 板上软件基线: 8×8 INT8 GEMV kernel 在 E203 上耗时 1630 cycles
- 板上硬件端到端: CPU 通过 MMIO 调用 GEMV 加速器耗时 243 cycles
- 端到端加速比: $1630/243 \approx 6.71 \times$
- RTL 推导硬件核心延迟: 8 个 COMPUTE 周期 + 1 个 WRDONE 周期, 共 9 cycles
- 外设基地址: `0x1004_1000`
- 上板结果: PL UART 输出显示 SW GEMV PASS 和 HW GEMV PASS

当前状态 软件仿真链路已经完成, 硬件加速器已经集成到 E203 SoC 并完成 FPGA 上板验证。BRAM 推断失败导致 CPU 卡死在 PC 0 的关键问题已经定位并修复。最终板端测试中, E203 通过 PL UART 输出完整软件和硬件 GEMV 结果, 8 个输出元素均与 golden values 一致, 并获得软件 baseline 与硬件端到端 cycle 数据。

1 项目背景与总体目标

1.1 背景

大语言模型推理中存在大量线性层计算，例如 Q/K/V projection、attention output projection 和 FFN 层。在线性层中，矩阵向量乘法（GEMV）和矩阵矩阵乘法（GEMM）占据主要计算量。为了降低存储带宽、访存能耗和乘加资源开销，实际推理系统通常使用 INT8 或更低比特量化数据。

本课程设计选择 INT8 GEMV 作为硬件加速对象。一方面，GEMV 算子形式简单，便于在 E203 软核系统中建立完整的正确性验证；另一方面，GEMV 具有清晰的数据复用和并行乘加结构，适合作为 FPGA 数据通路设计的入门加速模块。

1.2 目标

项目目标不是单独完成一个 RTL 模块，而是完成一条可运行的软硬件协同链路：

1. 在 WSL 中搭建 E203 iVerilog 软件仿真环境，确认用户 C 程序可以在 E203 RTL 模型上运行。
2. 实现 8×8 INT8 GEMV 软件基线，验证输出正确性并测量 E203 cycle 数。
3. 在 Vivado 工程中实现 GEMV hardware accelerator，并将其接入 E203 peripheral ICB bus。
4. 编写固件，通过 MMIO 配置加速器、启动计算、轮询完成状态并读取输出。
5. 在 FPGA 板端通过 ILA 和 PL UART 观察 CPU、加速器与程序输出，验证完整软硬件链路。

2 开发环境与工程结构

2.1 硬件与软件环境

本项目使用的目标开发板为 ALINX AXU3EG，FPGA 型号为 xczu3eg-sfvc784-1-i。主要工具和版本如表 1 所示。

表 1: 项目开发环境

工具或平台	版本或说明
FPGA 开发板	ALINX AXU3EG
FPGA 器件	xczu3eg-sfvc784-1-i
Vivado	2022.2
RISC-V GCC	Nuclei GCC 9.2.0
RTL 仿真	Icarus Verilog 12.0
SDK	hbird-sdk
板端调试	Vivado Hardware Manager + ILA, PL UART 115200 8N1

2.2 工程结构

课程提供的主要工程目录位于 `lecture2_0420_release/`。其中，Vivado 工程、固件工程和辅助工具目录如表 2 所示。

表 2: 主要工程目录

路径	作用
<code>lecture2_0420_release/trans/</code>	Vivado 2022.2 FPGA 工程
<code>trans.srcs/sources_1/new/</code>	新增 RTL 模块目录
<code>test1/application/main.c</code>	FPGA 固件主程序
<code>hexdump-2.1.0/</code>	hex/bin 转换和辅助脚本目录
<code>/home/cjt/csapp/e203_hbirdv2/</code>	WSL 中的 E203 iVerilog 仿真工程
<code>/home/cjt/csapp/hbird-sdk/</code>	WSL 中的 SDK 应用工程

3 软件基线设计与仿真结果

3.1 软件 GEMV kernel

软件基线程序实现固定尺寸 8×8 INT8 GEMV:

$$y_i = \sum_{j=0}^7 W_{i,j} x_j, \quad i = 0, 1, \dots, 7 \quad (3.1)$$

其中矩阵 W 和输入向量 x 使用 `int8_t`，累加器和输出 y 使用 `int32_t`。软件 kernel 采用双重循环实现，每个输出元素需要 8 次 signed 乘加，总共完成 64 次乘加操作。

该阶段的重点是建立可复现的软件基线：一方面确认 INT8 GEMV 的 C 程序能够在 E203 RTL 仿真环境中正确运行，另一方面测量纯软件实现的 cycle 数，为后

续硬件加速效果提供参考。

3.2 软件基线在整体框架中的作用

软件基线不是最终优化对象，而是整个设计框架中的参考坐标。它承担三个作用：第一，提供一组已经验证过的输入、输出和 **golden values**，保证后续硬件结果可以逐项比较；第二，给出 E203 顺序执行 INT8 GEMV 时的 **cycle** 数，作为加速比计算的分母；第三，保留完全由 C 程序实现的路径，便于区分“算法本身错误”和“硬件外设或总线接口错误”。

软件程序在 hbird-sdk 的 **baremetal** 环境中编译为 E203 可执行镜像，再加载到 E203 RTL 仿真模型中执行。报告中不展开具体命令，因为这些命令只属于工具链复现步骤；从设计角度看，关键是软件基线与硬件加速器使用同一类数据格式：矩阵和输入向量为 **int8_t**，输出为 **int32_t**，从而保证软硬件比较的含义一致。

仿真程序会打印每个输出元素，并将结果与 **golden values** 比较。早期软件基线结果如表 3 所示。

表 3: 8×8 INT8 GEMV 软件基线结果

输出元素	软件结果	Golden
y_0	56	56
y_1	-14	-14
y_2	25	25
y_3	-23	-23
y_4	33	33
y_5	24	24
y_6	36	36
y_7	-8	-8

E203 iVerilog 仿真输出显示 **INT8 GEMV PASS**，并测得早期软件基线：

$$T_{\text{software}} = 1638 \text{ cycles} \quad (3.2)$$

该结果作为当前项目的软件 **baseline**。

后续 FPGA 板端测试使用对角矩阵数据集，PL UART 实测软件 GEMV 为 1630 **cycles**。两个数值来自不同测试数据和运行环境，最终性能分析以板端 UART 输出的 1630 **cycles** 作为软件端到端对比基准。

4 硬件加速器设计

4.1 总体方案

硬件加速器被设计为 E203 SoC 的 memory-mapped ICB 外设。CPU 通过普通 load/store 指令访问外设寄存器，不需要修改 E203 core 本身。整体运行流程如下：

1. CPU 将 8×8 INT8 矩阵 W 写入外设内部 W_mem 。
2. CPU 将 8 维 INT8 向量 x 写入外设内部 x_mem 。
3. CPU 向 CTRL 寄存器写 start bit。
4. 加速器在硬件中执行 8 路并行 signed MAC。
5. 加速器置位 done bit, CPU 轮询后读取 Y_MEM。

该方式的优点是接口简单、调试直观，并且与 E203 原有 peripheral bus 结构兼容。

4.2 寄存器映射

加速器 RTL 文件为：

代码片段 1: GEMV 加速器 RTL 文件路径

```
lecture2_0420_release/trans/trans.srscs/sources_1/new/gemv_accel.v
```

外设基地址为 $0x1004_1000$ ，寄存器映射如表 4 所示。

表 4: GEMV 加速器 MMIO 寄存器映射

偏移地址	名称	说明
0x000	CTRL	bit[0] 为 start, bit[1] 为 done
0x010--0x04C	W_MEM	8×8 INT8 权重矩阵, row-major, 4 个 int8 打包为 1 个 word
0x050--0x054	X_MEM	8 维 INT8 输入向量, 4 个 int8 打包为 1 个 word
0x060--0x07C	Y_MEM	8 个 int32 输出元素, 只读
0x080	DEBUG_SW_CYCLES	固件写入的软件 GEMV cycle delta
0x084	DEBUG_HW_CYCLES	固件写入的硬件辅助端到端 cycle delta

4.3 计算数据通路

加速器内部包含 8 个 signed 32-bit accumulator，分别对应 8 个输出元素。每个计算周期处理一列输入：当前列索引为 `col`，硬件读取输入向量的第 `col` 项和权重矩阵第 `col` 列，并并行执行：

$$acc_i \leftarrow acc_i + W_{i,col} \times x_{col}, \quad i = 0, 1, \dots, 7 \quad (4.1)$$

FSM 状态如表 5 所示。

表 5: GEMV 加速器 FSM

状态	作用
IDLE	等待 CPU 写 start bit
COMPUTE	连续处理 8 列输入，每列执行 8 路并行 MAC
WRDONE	将 accumulator 写入 <code>y_mem[0..7]</code> 并置位 done

因此硬件核心计算延迟为：

$$T_{core} = 8 + 1 = 9 \text{ cycles} \quad (4.2)$$

与软件基线相比，核心计算理论加速比为：

$$S_{core} = \frac{1630}{9} \approx 181 \quad (4.3)$$

需要强调的是， T_{core} 只统计硬件内部乘加和写回结果的时间，不包含 CPU 写入矩阵、写入向量、启动外设、轮询 done 和读取结果的 MMIO 开销。因此最终展示时应同时给出硬件核心延迟和硬件辅助端到端延迟。

5 SoC 集成与固件设计

5.1 E203 外设总线集成

本设计没有修改 E203 core 内部流水线，而是把 GEMV 加速器作为一个 memory-mapped peripheral 接入 SoC。这样做的原因是接口边界清晰：CPU 仍然执行标准 load/store 指令，硬件加速器只需要遵守 E203 peripheral ICB bus 的握手协议即可。相比在执行级直接加入新指令，这种方式对原处理器侵入更小，也更适合课程工程中的 FPGA 验证。

原工程中 O14 slot 对应 `0x1004_1000--0x1004_1FFF` 地址窗口，原用途是 AXI example peripheral。本项目保留该地址窗口和总线路由关系，将其后端替换为直接 ICB slave 形式的 `gemv_accel`。因此 CPU 访问 `0x1004_1000` 起始的寄存器时，总线 fabric 会把请求送到 GEMV 外设，而不是送往原示例 AXI 外设。

这个集成方式形成了三层结构：E203 core 负责运行控制程序，ICB bus 负责地址译码和读写握手，gemv_accel 负责保存输入数据、执行乘加和返回输出。三层之间只通过寄存器读写交互，避免了软件和硬件实现细节互相耦合。

5.2 固件 MMIO 访问

固件是软硬件协同框架中的控制层。它不参与硬件内部 MAC 的实现，而是负责把软件中的矩阵和向量打包为 32-bit word，通过 MMIO 写入外设寄存器，然后写 CTRL.start 启动计算，轮询 CTRL.done，最后读取 8 个 32-bit 输出结果。

固件中定义了与硬件一致的寄存器地址。该定义相当于软硬件接口契约：只要寄存器偏移、数据打包顺序和状态位语义保持一致，CPU 侧代码和 RTL 内部实现就可以独立演进。

代码片段 2: 固件中的 GEMV MMIO 定义

```
#define GEMV_BASE      0x10041000UL
#define GEMV_CTRL      (*(volatile uint32_t *)(GEMV_BASE + 0x000))
#define GEMV_W_REG(k)  (*(volatile uint32_t *)(GEMV_BASE + 0x010 + (k)*4))
#define GEMV_X_REG(k)  (*(volatile uint32_t *)(GEMV_BASE + 0x050 + (k)*4))
#define GEMV_Y_REG(i)  (*(volatile uint32_t *)(GEMV_BASE + 0x060 + (i)*4))
#define GEMV_DBG_SW_CYCLES (*(volatile uint32_t *)(GEMV_BASE + 0x080))
#define GEMV_DBG_HW_CYCLES (*(volatile uint32_t *)(GEMV_BASE + 0x084))
```

当前板端验证使用对角矩阵测试数据：

$$W_{i,i} = i + 1, \quad x = \{1, 2, 3, 4, 5, 6, 7, 8\} \quad (5.1)$$

对应 golden output 为：

$$y = \{1, 4, 9, 16, 25, 36, 49, 64\} \quad (5.2)$$

选择对角矩阵是为了让每个输出元素都具有简单且不同的 golden value，便于 UART 日志和 ILA 观测时快速判断结果是否错位。固件先运行软件 GEMV 并记录 cycle delta，再运行硬件 GEMV 并记录包含 MMIO 访问的端到端 cycle delta。两个 cycle delta 会写入加速器 debug 寄存器，便于 ILA 观察。

6 程序镜像转换与关键问题修复

6.1 64-bit 程序镜像转换

E203 在 FPGA 工程中从 ITCM 取指，而 ITCM 的仿真 RAM 宽度为 64 bit。WSL SDK 生成的 .verilog memory image 则是 byte-per-token 格式。二者格式不匹配：如果直接读取 byte token，Vivado 会把每个 byte 当成一个 64-bit word，导致指令排列完全错位，CPU 取到的机器码不再是原始程序。

因此程序镜像转换不是附属工具步骤，而是系统启动链路的一部分。转换逻辑按 8 byte 为一组，将 byte stream 打包为 little-endian 的 64-bit word，使 \$readmemh 初始化后的 ITCM 内容与 E203 取指端口宽度一致。这样 CPU 才能从地址 0 开始按预期取到固件指令。

6.2 BRAM 推断失败问题

上板初期，CPU 长时间停留在 0x00000000，说明问题不在 GEMV 计算本身，而在更早的启动或取指链路。进一步检查综合网表后发现设计中没有推断出 BRAM，意味着 ITCM 和 DTCM 没有以真实片上存储的形式存在。CPU 从异常的 ITCM 中取到无效指令后触发 trap，最终回到 mtvec=0 并卡死在 PC 0。

根因是原始 sirv_sim_ram.v 使用 generate 循环展开出多个 always @(posedge clk) 块，每个字节通道一个写块。Vivado 对这种写法无法稳定推断 block RAM。这个问题说明，硬件框架设计不仅包括计算模块，还包括程序存储、复位、时钟和综合器可识别的存储器编码风格；其中任何一环失效，CPU 都无法运行到加速器访问阶段。

修复内容如下：

- 将所有字节通道写操作合并到同一个 always @(posedge clk) 块。
- 使用 [8*j +: 8] 完成变量 part-select，避免非法动态区间写法。
- 补充 genvar i；声明，使底部 generate 逻辑可以综合。
- 为 RAM 增加 (* ram_style = "block" *) 属性，辅助 Vivado 推断 BRAM。

修复后综合通过，Block RAM Final Mapping Report 中观察到的 BRAM 使用如表 6 所示。

表 6: BRAM 推断结果

模块	大小	RAMB36E2 数量
ITCM	8K×64	16
DTCM	16K×32	16
合计	-	32

该问题修复后，E203 可以从 ITCM 正常取指并执行固件。

7 上板调试与当前验证结果

7.1 ILA 配置

ILA 的作用不是替代 UART 输出，而是在硬件内部提供状态证据。早期调试阶段曾临时观察 E203 IFU 和寄存器堆，用于定位 CPU 是否正常取指；在 CPU 启动链路稳定后，当前工程保留 GEMV 外设内部 ILA，重点观察加速器的控制状态、列计数、命令触发和输出结果。

表 7: 当前 GEMV ILA 观测点

观测信号	位置	作用
ctrl_done	GEMV accelerator	判断硬件计算是否完成
state, col	GEMV accelerator	观察 IDLE/COMPUTE/WRDONE 状态和 8 列计算过程
cmd_fire, icb_cmd_addr	ICB slave interface	观察 CPU 是否真正访问 GEMV 地址窗口
dbg_hw_cycles, y_mem[0]	GEMV accelerator	关联端到端周期和第一个输出结果

GEMV 外设内部 ILA 当前连接如下，probe0 被打包为控制状态和调试计数，probe1 连接第一个输出元素：

代码片段 3: GEMV 外设 ILA probe

```
ila_0 u_ila_gemv (  
    .clk      (clk),  
    .probe0   ({ctrl_done, state, col, cmd_fire,  
                icb_cmd_addr[11:0], dbg_hw_cycles[12:0]}),  
    .probe1   (y_mem[0])  
);
```

7.2 PL UART 接出与串口配置

为获得完整输出结果，本项目将 E203 内部 UART0 通过 GPIOA 的 IOF 复用接到 FPGA 顶层，并绑定到 AXU3EG 板卡的 PL UART 管脚。E203 SDK 在启动阶段执行 gpio_iof_config(GPIOA, IOF_UART_MASK) 和 uart_init(SOC_DEBUG_UART, 115200)，因此固件中的 printf 会通过 UART0 输出。

UART0 的板端连接如表 8 所示。

表 8: PL UART 管脚连接

E203 信号	顶层端口	FPGA 管脚
GPIOA[16] / UART0 RX	uart0_rxd	AH11
GPIOA[17] / UART0 TX	uart0_txd	AH12

上板测试时使用板卡 PL UART 对应的 CP210x USB-UART 设备，Windows 中识别为 COM13。串口参数为 115200 8N1，关闭 flow control。UART 输出用于给出完整的结果列表和 cycle 数据，ILA 用于补充硬件内部状态，两者共同构成板端验证链路。

7.3 板端串口结果

早期 ILA 调试中，已经通过 GEMV ILA 观察到：

代码片段 4: 当前 ILA 观测结果

```
ctrl_done = 1
y_mem[0] = 0x00000001
```

最终板端测试改用 PL UART 输出完整运行日志。当前测试数据的 expected output 是：

$$y = \{1, 4, 9, 16, 25, 36, 49, 64\} \quad (7.1)$$

串口输出摘录如下：

代码片段 5: PL UART 板端输出摘录

```
HummingBird SDK Build Time: May  4 2026, 16:37:03
Download Mode: ILM
CPU Frequency 15991767 Hz
INT8 GEMV Accelerator Demo
Matrix shape: 8 x 8

SW y[0] = 1, golden = 1
SW y[1] = 4, golden = 4
SW y[2] = 9, golden = 9
SW y[3] = 16, golden = 16
SW y[4] = 25, golden = 25
SW y[5] = 36, golden = 36
SW y[6] = 49, golden = 49
SW y[7] = 64, golden = 64
SW GEMV PASS
SW Cycle delta: 1630
```

```
1 HW y[0] = 1, golden = 1
1 HW y[1] = 4, golden = 4
2 HW y[2] = 9, golden = 9
2 HW y[3] = 16, golden = 16
2 HW y[4] = 25, golden = 25
2 HW y[5] = 36, golden = 36
2 HW y[6] = 49, golden = 49
2 HW y[7] = 64, golden = 64
2 HW GEMV PASS
2 HW Cycle delta: 243
```

该结果说明：

- E203 固件已经能够运行到访问 GEMV 外设的位置。
- CPU 可以通过 MMIO 访问 `0x1004_1000` 地址段。
- CPU 已经成功写入数据并启动加速器。
- 加速器已经完成计算，并产生了完整正确的 8 个输出元素。
- 软件 GEMV 板端执行周期为 1630 cycles。
- 硬件加速器端到端调用周期为 243 cycles。

这表明从 CPU 到外设的控制链路、数据链路和串口观测链路均已打通。ILA 结果用于证明硬件内部状态确实发生变化，PL UART 结果用于给出完整正确性和性能数据。

7.4 Reset 与时钟设计

旧设计中板上 reset 会复位 MMCM，导致 ILA/debug hub 依赖的 `clk_16M` 停止，Vivado Hardware Manager 可能报无法连接 debug core。当前已修改 reset 方案：

- MMCM `resetn` 固定为 `1'b1`，不再由板上 reset 拉停。
- CPU reset 改为由 `mmcm_locked` 和上电延迟释放逻辑控制。
- 上电延迟计数宽度为 20 bit，在 16 MHz 下约为 65 ms。

这个修改体现了板端调试时的一个设计原则：调试基础设施依赖的时钟不能被普通业务 reset 随意关闭。当前设计将 MMCM 保持运行，只对 CPU 复位释放做延迟控制，从而避免 reset 操作破坏 ILA/debug hub 连接。

8 性能分析

8.1 核心计算与端到端性能

软件基线、硬件端到端调用和 RTL 核心计算周期对比如表 9 所示。其中，软件和硬件端到端数据来自 PL UART 板端输出；RTL 核心计算延迟来自 `gemv_accel.v` 的 FSM 结构推导。

表 9: 软件、硬件端到端与 RTL 核心周期对比

实现方式	计算内容	周期数
E203 软件 kernel	8×8 INT8 GEMV 软件双重循环	1630 cycles
GEMV 硬件端到端	写入 W/x 、启动、轮询 done、读取 y	243 cycles
GEMV RTL 核心	8 路并行 MAC 计算与结果写回	9 cycles

硬件端到端加速比为：

$$S_{\text{end-to-end}} = \frac{1630}{243} \approx 6.71 \quad (8.1)$$

硬件核心只需要 9 cycles 的原因是它将 8 个输出元素的 MAC 并行展开，每个周期处理一个输入列。软件实现虽然也只计算 64 次乘加，但每次乘加、循环控制、访存和寄存器调度都由 E203 顺序执行，因此 cycle 数明显更高。

需要注意，9 cycles 是 RTL 状态机推导出的核心计算延迟，并非板上串口直接测得的端到端周期。板上实测的 243 cycles 更能反映 CPU 实际调用加速器时的系统级开销。

8.2 端到端开销

实际系统中，CPU 使用硬件加速器时不仅要等待核心计算，还需要完成以下操作：

- 写入 16 个 32-bit word 的矩阵数据。
- 写入 2 个 32-bit word 的输入向量。
- 写 CTRL 启动外设。
- 轮询 CTRL.done。
- 读取 8 个 32-bit 输出结果。

因此硬件辅助端到端周期一定大于 9 cycles。最终报告中将三类指标分开呈现：

1. 软件 GEMV kernel cycle: 1630 cycles。

2. GEMV RTL 核心计算 cycle: 9 cycles, 由 FSM 推导。
3. CPU + MMIO + GEMV 外设的硬件辅助端到端 cycle: 243 cycles。

从端到端结果看, 硬件加速器虽然核心计算只需 9 cycles, 但一次调用中仍包含 16 次权重写入、2 次输入向量写入、1 次 start 写入、done 轮询和 8 次输出读取。对于 8×8 这样较小的矩阵, MMIO 访问开销占比较高, 因此端到端加速比低于核心计算理论加速比。若矩阵规模扩大或采用批量数据搬运方式, 硬件核心并行度带来的收益会更明显。

9 当前限制与后续工作

目前项目已经完成主体实现和完整上板验证, 但仍存在以下可改进点:

1. 若需要更强的板上波形证据, 可使用当前 GEMV ILA 抓取 state、col 和 ctrl_done, 展示 8 个 COMPUTE 周期和 1 个 WRDONE 周期。
2. 当前 GEMV 已将 8-bit operand sign-extend 到 16-bit 后再相乘, 后续可继续约束乘法器资源映射, 降低综合资源和综合时间。
3. 当前工程只保留 GEMV 相关 ILA。若最终演示完全依赖 UART, 可以进一步生成无 ILA 的发布 bitstream, 减少 debug 资源占用。
4. 当前测试规模为 8×8 , 适合验证链路。后续可扩展为更大矩阵或 tile-based GEMV, 以提高硬件计算时间在端到端开销中的占比。

10 结论

本项目围绕 INT8 GEMV 算子, 完成了 E203 软件仿真、FPGA 工程迁移、硬件加速器设计、SoC 外设集成和完整上板验证。软件阶段建立了 8×8 INT8 GEMV 的正确性和性能基线, 早期 iVerilog 仿真测得软件 kernel 为 1638 cycles, 最终板端 UART 测得软件 GEMV 为 1630 cycles。硬件阶段实现了一个挂接在 E203 peripheral ICB bus 上的 memory-mapped GEMV 加速器, 采用 8 路并行 signed MAC, 可在 RTL 核心中以 9 cycles 完成 8×8 GEMV 的核心计算。

调试过程中, 项目定位并修复了 sirv_sim_ram.v 无法被 Vivado 推断为 BRAM 的关键问题, 使 E203 能够在 FPGA 上从 ITCM 正常取指运行。早期 ILA 证明 CPU 可以通过 MMIO 启动 GEMV 外设, 最终 PL UART 输出进一步证明 8 个硬件输出元素全部与 golden values 一致, SW GEMV PASS 和 HW GEMV PASS 均成立。板端实测硬件端到端调用周期为 243 cycles, 相比软件 1630 cycles 达到约 $6.71 \times$ 端到端加速。